IT University of Copenhagen
Payam Zahadat, paza@itu.dk

October 30, 2026

Advanced Topics in Robotics —Swarm Robotics

Session 1

Topics

Swarm robotics and collective intelligence; collective behaviors in nature and example bio-inspired algorithms; discussion on some enablers and driving mechanisms of self-organized collective intelligence. You will work on an algorithm that makes the swarm disperse.

Literature (recommended to read before class)

I assume that all literature material listed has been read *before* the classes. The classes will take the form of a brief introduction to the topic and then a wider discussion. Hence, do not expect that all material will be covered in class.

SwarmRoboticsDorigo2021 [Swarm Robotics: Past, Present, and Future]: This paper provides an introductory overview on swarm robotics, including some history, key concepts, challenges, and future directions.

Good Experimental Methodology, Bonsignorio et al., pp. 3-4 This is a paper on good experimental practices in robotics. Note, that we emphasize only two pages, but the rest may be useful in your future projects.

DispersionUgur2007 [Dispersion of a swarm of robots based on realistic wireless intensity signals]: Here, a dispersion algorithm is implemented in a physics-based simulator of a real robot platform. In addition to noise, a key challenge was the lack of directional information about the signals, making it impossible to implement a typical potential field algorithm. Read that to learn about example challenges in a real robot experiment and examples on how to approach them.

Basic statistics. We need to understand how a robot performs and for this we use measurements and basic statistics. Please refresh your knowledge of why we use statistics, frequency plots, histograms, box-and-whisker plots, averages, standard deviation, etc. A good introduction is "Elementary Statistics - A Step by Step Approach, Allan G. Bluman", chapters 1-3, but if you had a book earlier in your studies, that is likely to be suitable as well.

Assignments

For the swarm robotics module, you will work in groups of two or three. You will use a simple kinematic simulator (no physics) that I provide on LearnIT, and you will extend it for your assignments. Download the simulator code and read the accompanying documentation. Then run the code, familiarize yourself with the implemented robot, and try controlling the robots with a simple behavior.

Report

Part of your exam report at the end of the semester will be based on your work in this module. You will implement two main behaviors: dispersion and flocking. The details will be introduced gradually over the next sessions.
Your report for this module has two clearly marked parts:

- **Group part:** a brief description of the behaviors and your implemented algorithms (explain how they work in an understandable way, not your code; diagrams can help); your design choices and assumptions; your experimental setup, evaluation methods and performance metrics; and the results (figures, statistics).

- **Individual part:** your own discussion and reflection on why things happened as they did: what your results mean and the insights you gained, the reasoning behind your choices, other possible solutions, the real-world applicability of your method, and suggestions for future investigations.

Whatever you explore (e.g., algorithm parameters, swarm size, noise), describe what you did and found in the group part, and discuss what it means in your individual part. See the introduction lecture of the course for a more detailed description.

For this module, you also need to make a video

Along with your written report, you also need to upload a short **video** of your swarm's behavior (feel free to capture a selected nice part, or a part you find interesting). There is no need to make a professional-looking video. You can simply use your phone to film the behavior on your screen. Keep the video short (max 2 minutes in total for the module).

Note: For this module, we are particularly interested in your approach to experiment evaluation. Your system has many parameters, including algorithm parameters that you tune during development, robot-related parameters (e.g., sensor range, sensor noise, motion noise), and swarm or environment parameters (e.g., swarm size in a given arena, presence and configuration of light or obstacles). We expect you to scientifically study the effects of some of these parameters and present the results in a meaningful way (e.g., with figures and statistics). Such a scientific study may require designing your experiment carefully. It may also involve formulating hypotheses (e.g., what you expect to happen if you change a certain parameter or part of the algorithm) and then testing whether those expectations hold true. Make sure to reflect on what you are testing and what the results mean.

Real-World Considerations

While the simulator abstracts away many real-world complexities, it is important to reflect on what could go wrong in a physical robot system that is not captured here. Think about:

- Limitations in sensing (e.g., blind spots, noise, delays, shadows and occlusions)
- Embodiment of the robots (e.g., collisions, physical constraints, positioning of components such as sensors and actuators, dynamics)
- Hardware constraints (e.g., computation time, battery life)
- Imperfections in communication, if you have any (e.g., message loss, interference, limited bandwidth)
- Concurrency and parallelism (e.g., timing issues, message timing mismatches); this is partly modelled in the simulator (see *Time and Control Cycles* in the documentation)
- Anything else?

Also think about what "advantages" real-world robots gain from physical embodiment, such as passive stability due to their design or natural feedback from interactions with the environment. For example, friction, contact, and body shape may help a robot reorient itself or escape tight spaces without explicit programming.

Noise Sensitivity

The simulator includes configurable noise models that are currently disabled by default (e.g., RAB bearing noise and message dropout, motion noise in position and heading updates, light and orientation sensing noise). You are encouraged to experiment with these noise settings to assess the robustness of your algorithm. How sensitive is your behavior to sensing or motion imperfections?

Mini-exercise

Try to get multiple robots moving randomly without colliding with walls or each other (note that the simulator does not model physical collisions: robots can overlap each other, so it is up to your controller to keep a safe distance). You must implement this behavior within each robot using only the sensors and motion model provided by the simulator. Do not introduce new sensing or motion capabilities beyond what is available.

Important hint: Always use the KISS[1] principle.

Assignment Part 1: Dispersion Behavior

The goal of this assignment is to implement a decentralized algorithm that causes a large number of robots to disperse throughout the arena. Since the arena is enclosed and relatively small, a purely repulsive behavior may be sufficient to achieve

[1] Keep it simple stupid.

dispersion. However, you are free to explore more refined strategies that combine both repulsion and weak attraction to maintain connectivity between robots.
Extension (Light modulation) Add one or more light sources, and modulate the behavior based on light intensity (for example, denser dispersion in brighter areas).
Note: You must implement this behavior using only the sensors and motion model provided by the simulator. **Do not** introduce new sensing or motion capabilities beyond what is available.
Note: The performance of your algorithm may depend on various parameters (e.g., swarm size, sensor ranges). Modify some of these parameters and analyze how they affect the behavior.
Note: You need to evaluate your dispersion behavior quantitatively. To do this, you must:

- Define one or more performance metrics to measure the effectiveness of the dispersion.
- Run experiments using these metrics and summarize the results. The functions `compute_metrics()`, `log_metrics()`, `logging_init()` and `logging_close()` in the code are placeholders where you can compute and log your metrics.
- Remember that you need to perform multiple runs and compute statistics (e.g., frequency, mean, standard deviation, or median and quartiles). **Why is this important?** Think about the role of randomness and stochasticity in evaluating behaviors.
- The random seed is fixed (`RANDOM_SEED`, at the top of the code). This has two consequences:
  - If you **restart the program** for each experiment, every experiment starts from the same initial configuration and gives exactly the same result (including any noise). In that case, change `RANDOM_SEED` before each experiment.
  - If you run **several experiments one after another within the same program** (e.g., in a loop), each experiment gets different random values, so they differ automatically.

  In both cases, your results stay reproducible: running the same setup again gives the same results.
- Running in headless mode (press `Space`) makes the runs much faster.